Library Management System (LMS) - Capstone Project

Full Stack Web Development Project

Presented by: Wong Heng, Kelvin
Date: 15 Dec 2025

Developer is an experienced hosting operations, infrastructure automation and cloud engineer learning full stack web development.

LMS is a web application designed to streamline library operations and enhance user experience

Project Overview

- Purpose of LMS is to will allow librarians to efficiently manage books, members, and borrowing/returning processes. Members will be able to search for books and view their borrowing history
- The two main user roles are Librarian (Admin) and Member
- The project is built using Spring Boot (Backend), React (Frontend), and MySQL (Database).

Project Goals

- Develop a fully functional web application using Spring Boot, React, and MySQL.
- Enable librarians to easily manage books and members.
- Provide a user-friendly interface for members to search for books and track their borrowing history.
- Create a robust and scalable system that can handle a moderate number of books and members.

Key Features

- User Management
  - User registration and login
  - Two user roles: Librarian (Admin) and Member
  - Profile management
  - Password reset functionality
- Member Management:
  - Register new members with personal details (name, address, contact information, etc.).
  - Edit member information.
  - Delete members from the system.
  - Search for members by name or ID.

Key Features (cont.)

- Book Management
  - Add, update, delete, and view books
  - Book details: ISBN, title, author, category, publication year, copies available
  - Search and filter books by various criteria
  - Track book status (available, borrowed, reserved)
- Lending Management
  - Borrow and return books
  - View borrowing history
  - Reserve books
  - Due date tracking
  - Automatic fine calculation

Business Rules

- Membership Rules
  - Members can borrow a maximum of 3 books at a time
  - Membership valid for 1 year from registration
  - Members must have valid membership to borrow books
- Lending Rules
  - Loan duration: 14 days
  - Maximum 2 renewals per book
  - Cannot borrow if having overdue books
  - Can not borrow if accumulated fines exceed $10
- Fine Calculation
  - $0.50 per day for overdue books
  - Fine starts accumulating from day after due date
  - Maximum fine per book: $20

Technical Requirements

- Backend
  - Spring Boot framework
  - RESTful API architecture
  - Spring Security for authentication
  - JPA/Hibernate for database operations
- Frontend
  - React.js
  - React Router for navigation
  - React Hooks for state management
  - Bootstrap or Material-UI for styling
  - Axios for API communication
- Database
  - MySQL database
  - Proper ER mapping
  - Referential integrity
  - Indexing
  - Data Validation

System Architecture

- The library system is a three-tier web application:
  - **Frontend**: React single-page application (built with Vite and TypeScript)
  - **Backend**: Spring Boot 3 (Java 21) providing a REST JSON API
  - **Database**: MySQL 8
- The browser talks only to the Spring Boot backend via normal HTTP requests. No server-side rendering, no WebSockets – just clean REST.

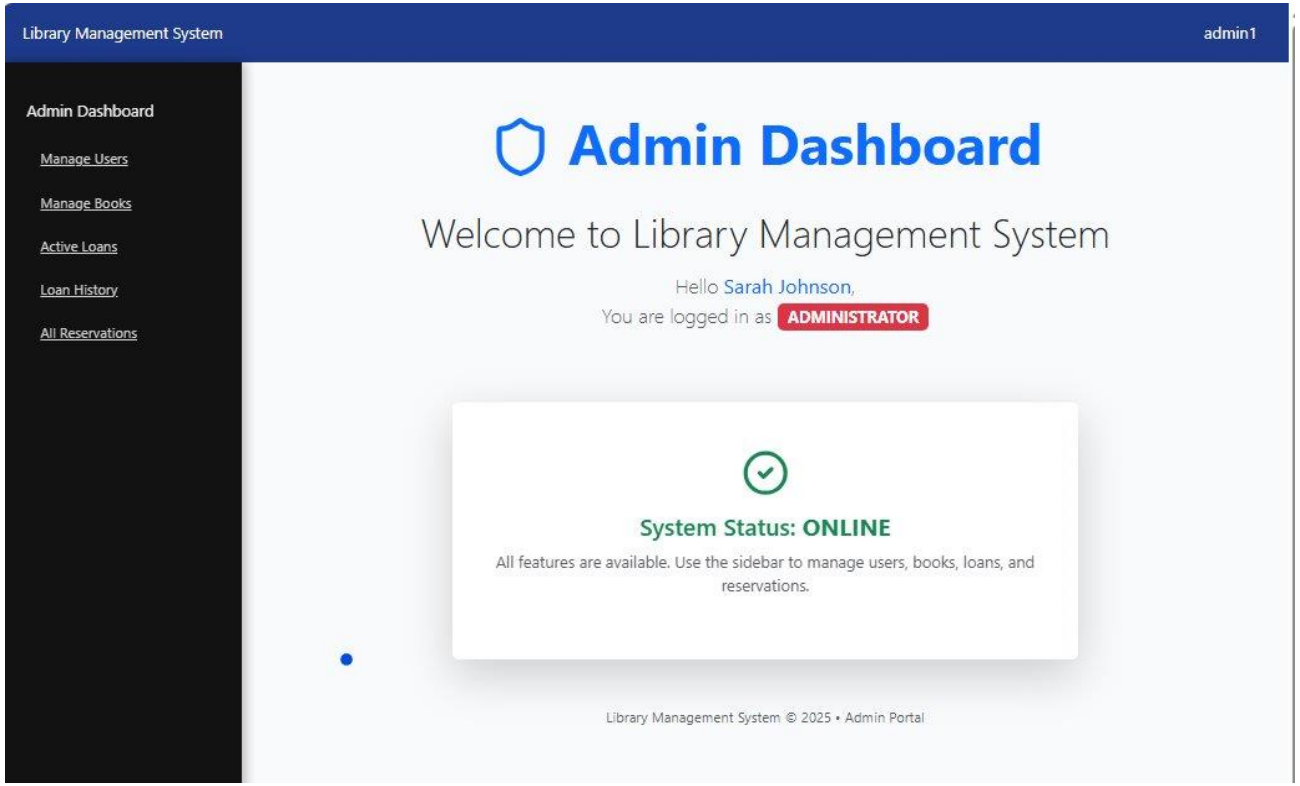
System Architecture (cont.)

- **MVC (Model-View-Controller) pattern is chosen because:**
  - Keeps concerns clearly separated (controllers handle HTTP, services contain business rules, repositories talk to the database)
  - Makes the code easy to understand and navigate – anyone can find the borrow logic quickly
  - Allows us to write fast, isolated unit tests for the service layer
  - Reuses the same business logic across multiple endpoints (borrow now, reserve, renew, etc.)
  - Feels familiar to most Java developers and has worked reliably for years

Screenshots of key features

Admin Portal

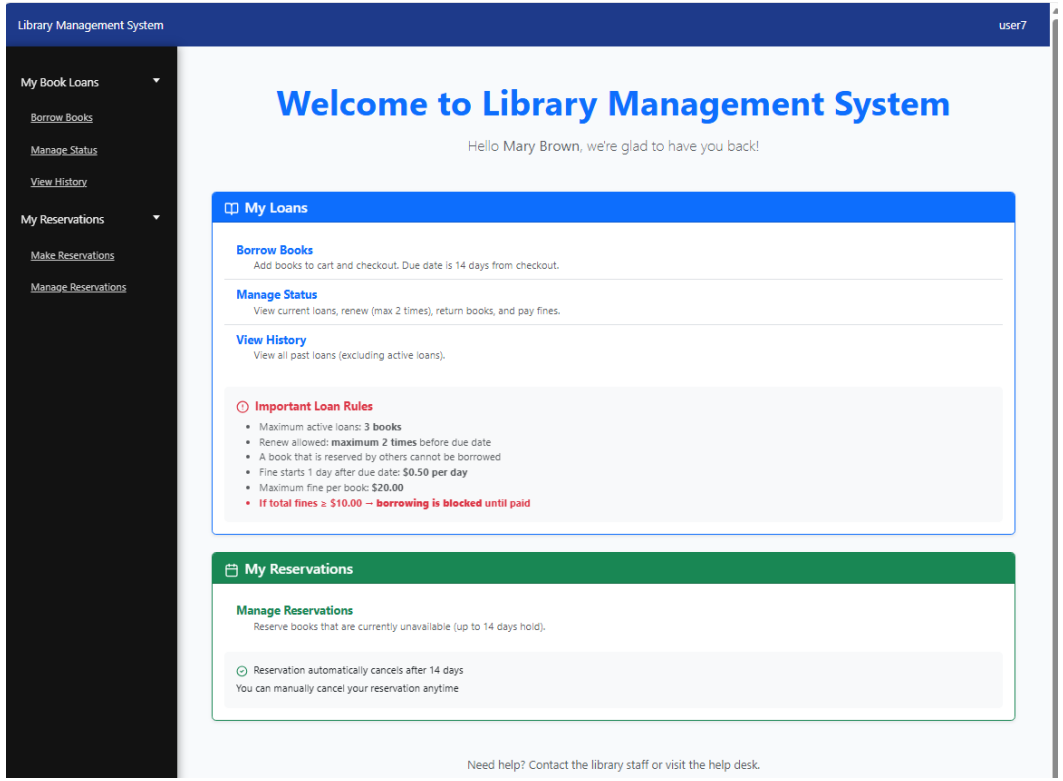
- Manage Users - CRUD ops on users
- Manage Books - CRUD ops on books
- Active Loans - RUD (no C) ops on books
- Loan History - View loan history
- Manage Reservations - RUD (no C) ops on reservations



Screenshot of key features (cont.)

Member Portal

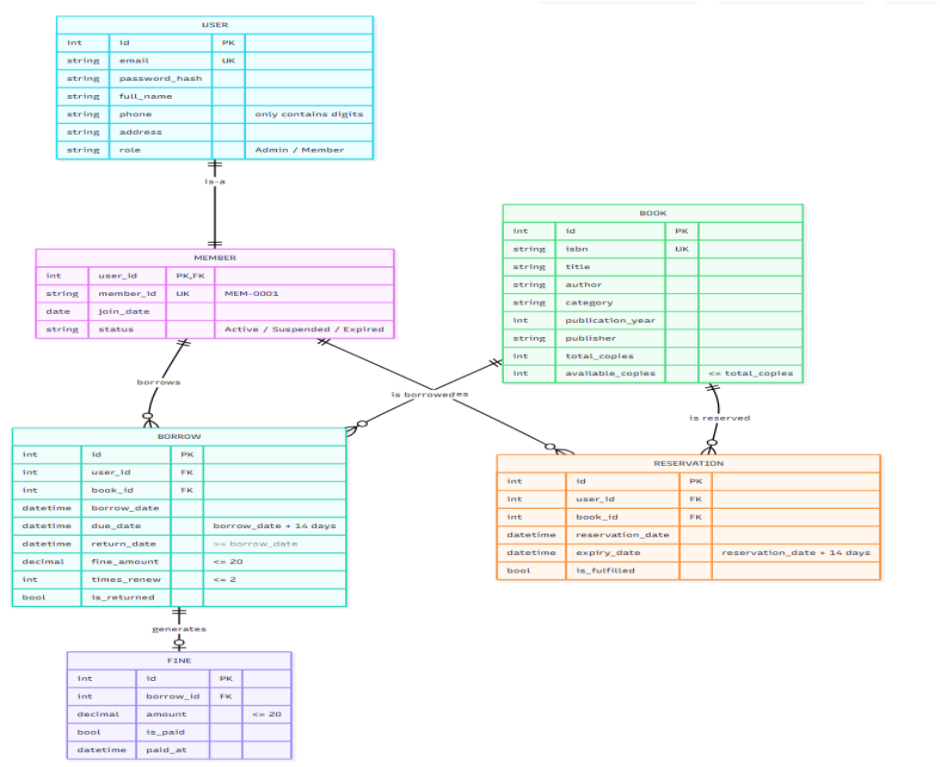
- My Loans
  - Borrow Books - Members can borrow books
  - Manage Status - Members can view active loans status, return, renew and/or pay fines on overdue books
  - View History - Member can view the loans history
- My Reservations
  - Make Reservations - Members can reserve books
  - Manage Reservations - Members view and cancel active reservations



Database & System Designs

ERD Diagram

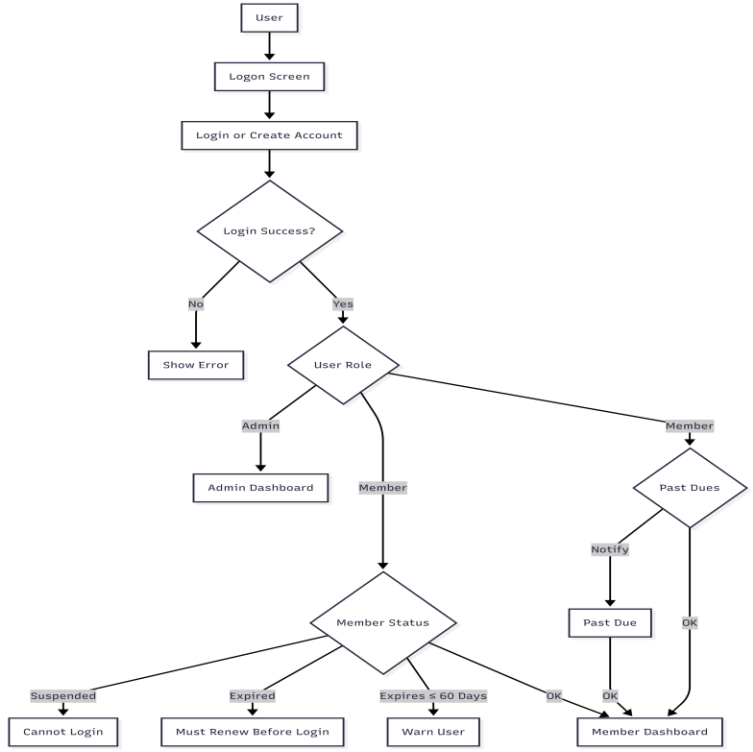
- A member borrows one or more borrowings (a member can borrow multiple items).
- A member reserves zero or more reservations (a member can make multiple reservations).
- Every member is a user, but not every user is necessarily a member
- A book can be borrowed in one or more borrowings (a book can be borrowed multiple times over its lifetime).
- A book can be reserved in zero or more reservations (a book can have multiple reservations).
- A borrowing can generate zero or one fine (a borrowing may result in a fine, but not all borrowings have fines).



Database & System Designs (cont.)

Wireframe feature diagram – Login

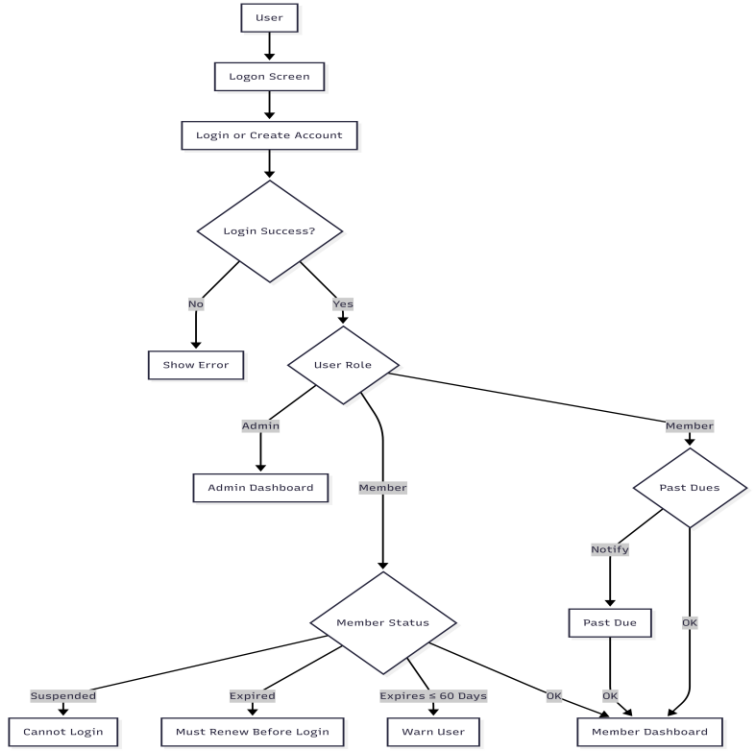
- When a user opens the application, they first see the **Logon Screen**, which offers two options: **Login** or **Create Account**.
- After submitting credentials:
- If login fails, an error message is displayed and the user stays on the logon screen.
- If login succeeds, the system checks the user's role and status.
- **For Admin users** They are taken directly to the **Admin Dashboard**.



Database & System Designs (cont.)

Wireframe feature diagram – Login (cont.)

- **For Member users** The system performs a series of checks in this order:
- **Membership Status Check**
  - If membership is **Suspended**, the user cannot log in and sees a message explaining the suspension.
  - If membership is **Expired**, the user is blocked from logging in and is informed they must renew their membership first.
- **Past Due / Overdue Books Check**
  - If the member has overdue books, a **Past Due notification** is shown, but they are still allowed to proceed to the dashboard.
- **Membership Expiry Warning**
  - If membership expires in 60 days or less, a non-blocking warning is displayed.
  - After all checks are complete and no blocking conditions exist, the member is directed to the **Member Dashboard**.



Database & System Designs (cont.)

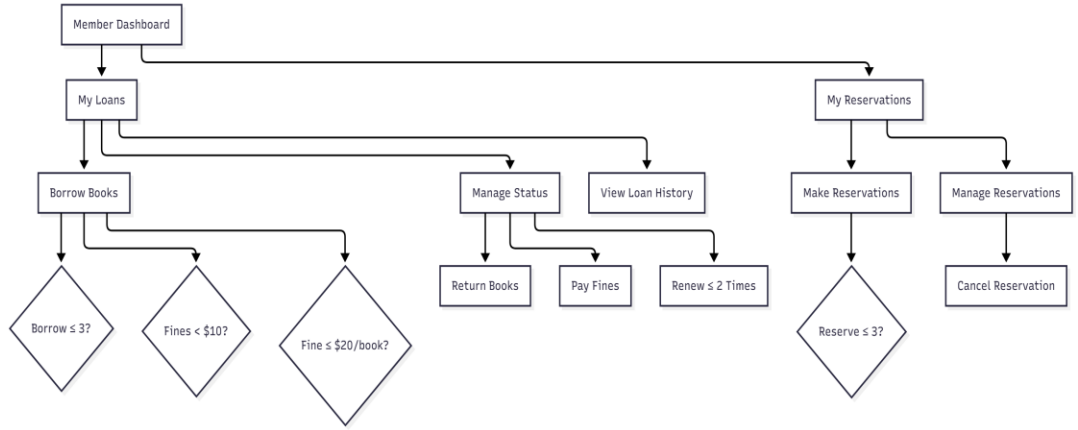
Member Dashboard Navigation & Rules

From the **Member Dashboard**, the user has two main sections:

1. My Loans

This section lets members manage their currently borrowed books.

- **Borrow Books** When a member tries to borrow a new book, the system checks three rules in sequence:
    - Can the member have more loans? (Maximum 3 active loans)
    - Are total unpaid fines below $10?
    - Is the fine on any single book $20 or less? Only if all three conditions are met can the borrow succeed.



Database & System Designs (cont.)

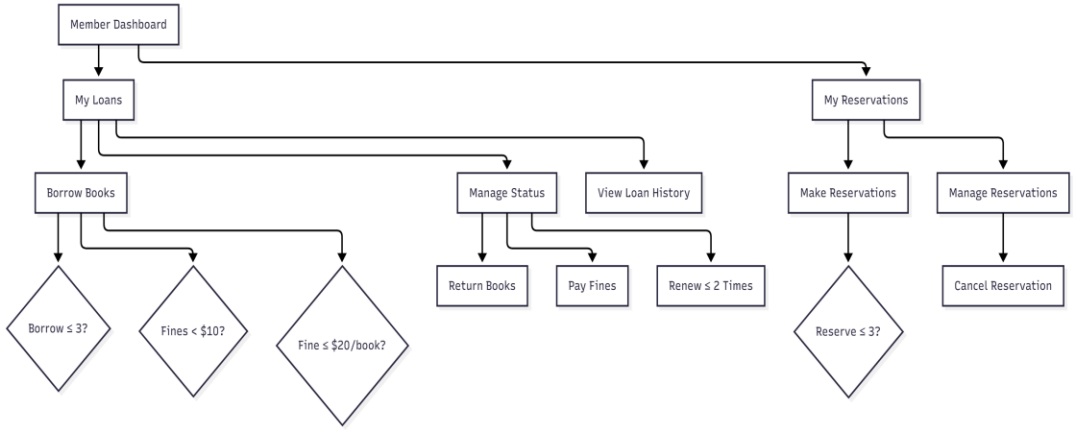
Member Dashboard Navigation & Rules (cont.)

- **Manage Status** From here members can:
  - Return borrowed books
  - Pay outstanding fines
  - Renew a loan (allowed up to 2 times per book)
- **View Loan History** Shows all past and current loans with dates and status.

2. My Reservations

This section handles book reservations.

- **Make Reservations** Members can reserve available books, but limited to a maximum of 3 active reservations at a time.
- **Manage Reservations** Allows members to cancel any of their existing reservations.

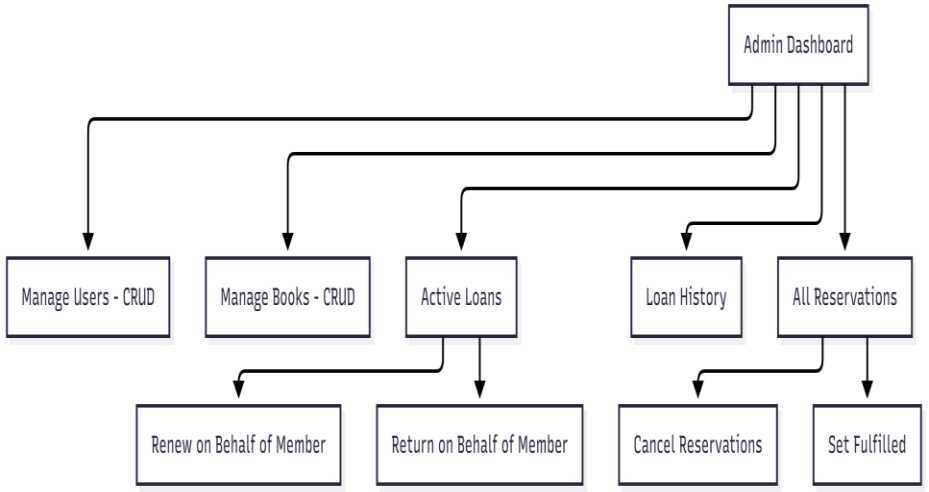


Database & System Designs (cont.)

Admin Dashboard Navigation & Functions

From the **Admin Dashboard**, administrators have full control over the system through these main sections:

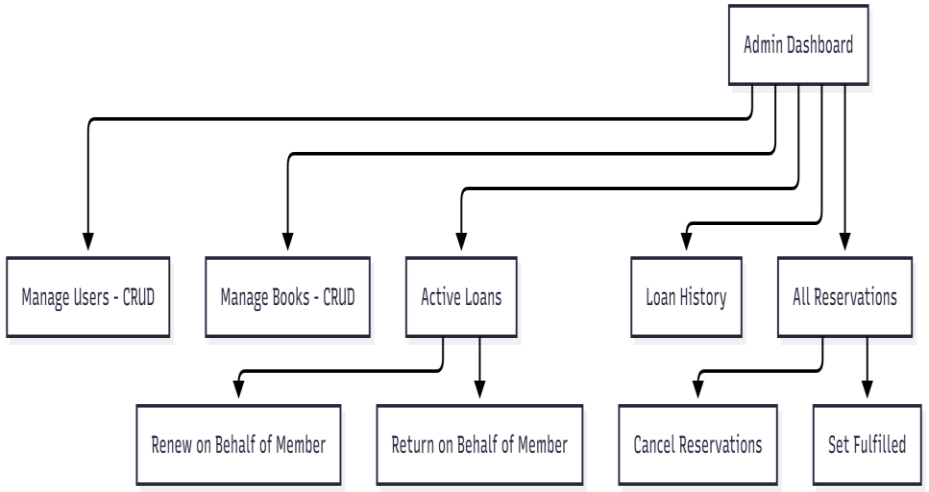
- **Manage Users** Full CRUD operations: create new member accounts, view, update, or suspend/activate existing users.
- **Manage Books** Full CRUD for the library catalog: add new books, edit details (title, author, copies, etc.), or remove/archive titles.



Database & System Designs (cont.)

Admin Dashboard Navigation & Functions (cont.)

- **Active Loans** View all current loans across all members. From here admins can:
    - Renew a loan on behalf of a member (even if the member has reached their personal renewal limit)
    - Return a book on behalf of a member (e.g., if the book was returned in person at the desk)
- **Loan History** Complete audit trail of every loan and return in the system, searchable and filterable.
- **All Reservations** View every active reservation. Admins can:
    - Cancel any reservation (e.g., if a book is damaged or policy changes)
    - Mark a reservation as fulfilled (when the member picks up the book)

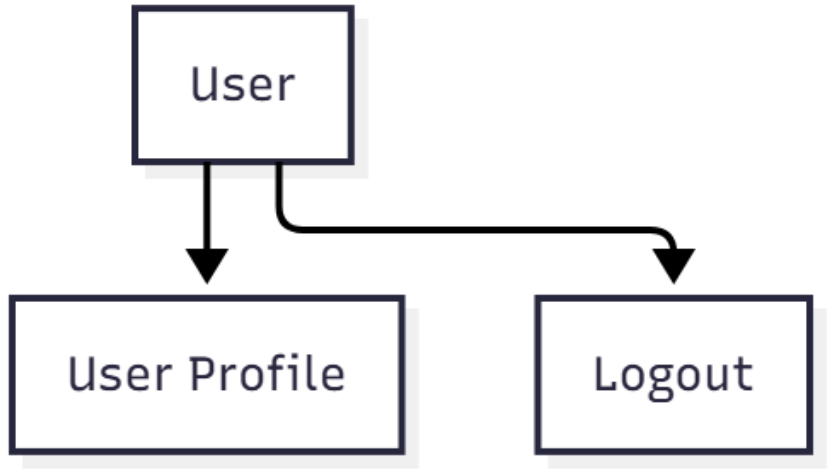


Database & System Designs (cont.)

Shared User Options

From any screen in the application (after successful login), both **Members** and **Admins** always have quick access to two common actions:

- **User Profile** Takes the user to a page where they can view and edit their personal details (name, contact info, etc.). For members, this may also show membership expiry date and status.
- **Logout** Immediately ends the session and returns the user to the Logon Screen.



API Endpoints

- Books API
  - /api/books - CRUD actions on book
- Authentication & Users API
  - /api/users - CRUD actions on users
- Members API
  - /api/members - CRUD actions on members
- Borrowing API
  - /api/borrows - CRUD actions on loans
- Reservations API
  - /api/reservations - CRUD actions on reservations
- Detailed reference -> LMS REST API Document.pdf

Test Cases - UATs

- UAT test cases includes
  - New user registration
  - Member logging in with overdue books
  - Member borrowing a book
  - Member returning a book
  - Member with overdue books and fines to pay
  - Member making a book reservation
  - Admin changing member status

Test Cases - UATs

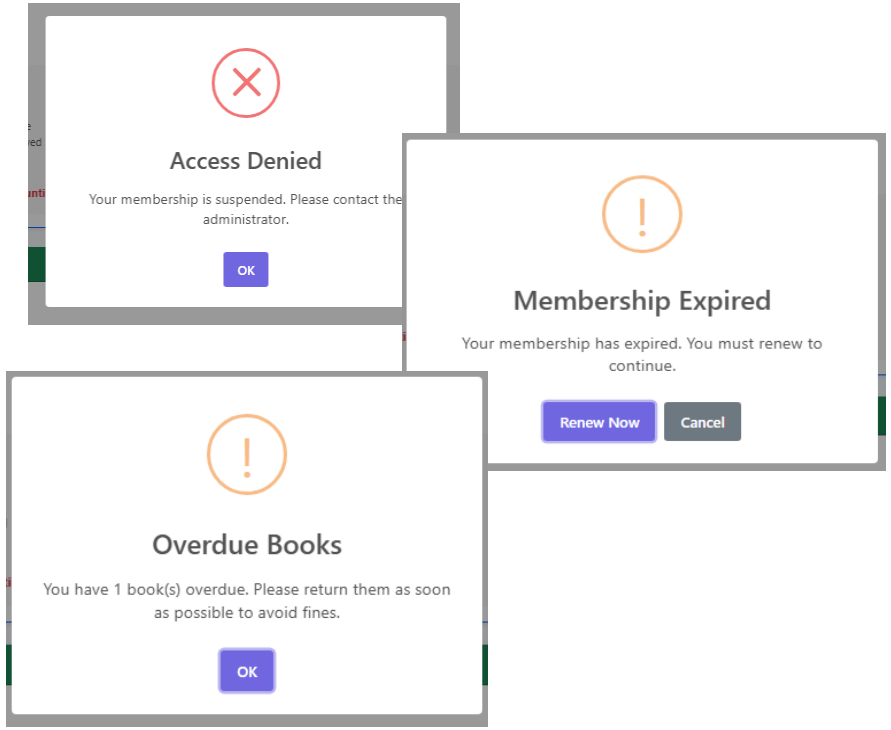
- Unit Test Cases
  - Test cases are created for Springboot java classes
  - Reference: Unit and Functional Test Cases.pdf
- Function Test Cases
  - Test cases are created for React/javascript using mocks
  - Reference: Unit and Functional Test Cases.pdf
- Functional UAT Test cases
  - Functional user test cases with screenshots
  - Reference: Funcational UAT Test Cases.pdf

Test Cases - UATs

- Unit Test Cases
  - Test cases are created for Springboot java classes
  - Reference: Unit and Functional Test Cases.pdf
- Function Test Cases
  - Test cases are created for React/javascript using mocks
  - Reference: Unit and Functional Test Cases.pdf
- Functional UAT Test cases
  - Functional user test cases with screenshots
  - Reference: Funcational UAT Test Cases.pdf

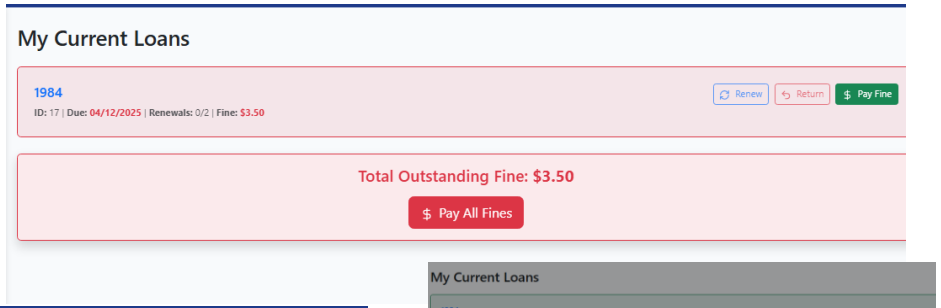
Test Cases – UATs (cont.)

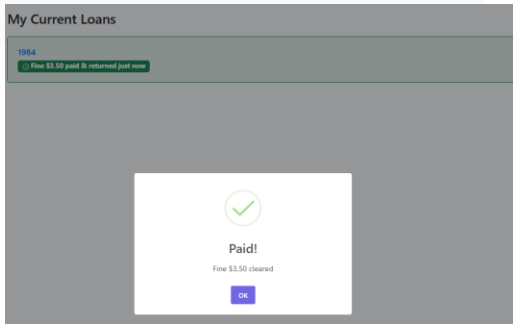
- Functional UAT Test cases (examples)
  - Member with Suspended status logs in
  - Member with Expired status logs in
  - Member with overdue books logs in

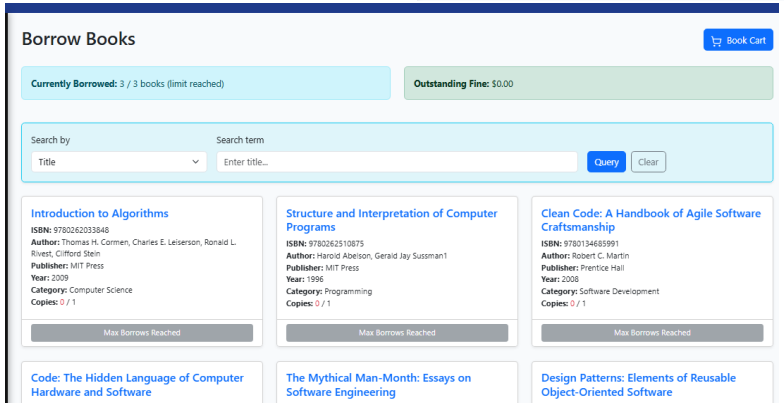


Test Cases – UATs (cont.)

- Cont.
  - Member pays fine
  - Member tries to borrow > 3 books

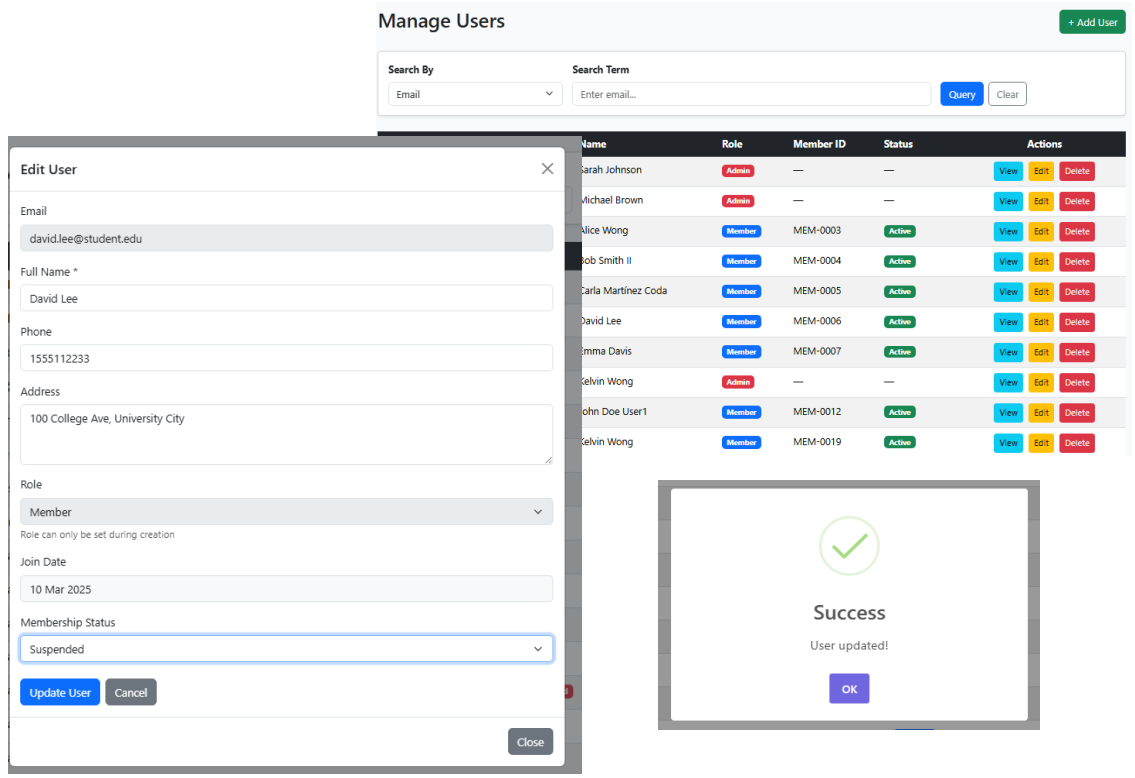






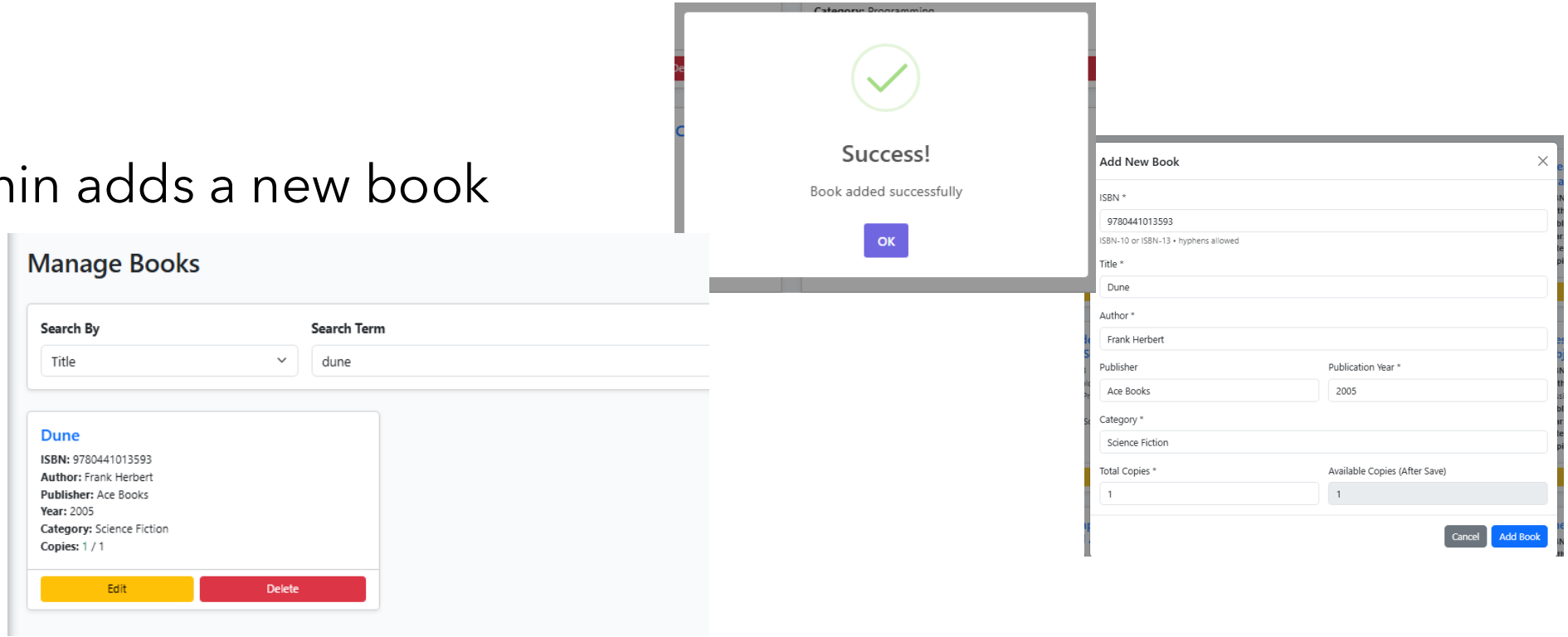
Test Cases – UATs (cont.)

- Cont.
  - Admin updates member status to Suspended



Test Cases – UATs (cont.)

- Cont.
  - Admin adds a new book



Challenges and Solutions

Challenges	Solutions
Keeping a consistent UI Design and experience	Mock up a quick protocol, address any UI and experience consistency issues, nail down design elements
Initial database design was too complicated, adding too many constraints and checks in the database, this overly encapsulate the logic from the code making debugging difficult	Redesigned the database such that only very important constraints and check are added to main integrity. Move less crucial (non-breaking) constraints and checks to backend and frontend codes making the overall code easier to understand
Mismatch between expected logic and code execution, e.g. a unique constraint was initially created to block members from borrowing the same book if already borrowed by same member keep failing	The issue was due to how the constraint was done. After various tried, found that using Boolean which accepts NULL fixes the issue after hours of trial and error
Packages and components used in nodejs 24.11.x don't work for testing environment, e.g. react-router-dom and axios	Needed to downgrade nodejs to 20.x, react-route-dom to 6x and custom json properties for axios for testing to work
Mocking Swal.fire elements failed constantly in test due to edge cases, losing a lot of hours in testing	Solution was to mock swal.fire globally and locally with a solution that also troubled many developers

Future Enhancements

- Add security features to database, REST endpoint and React login, add vault to store secure credentials.
- Add payment gateways for fines payment
- Add SMS/Email notifications for various matters like system maintenance, overdue books, reserved books is available.
- Add connectivity to external libraries when books are not available in the system and suggest other libraries.
- Link book reservations with borrow services, currently its manually resolved.
- Redesign application for microservice architecture to allow application to scale with increased load
- Enhance UI for use with mobile phones and tablets.
- Create metrics to understand popular books, highlight troublesome members (e.g. consistent late/overdue returns) and popular search terms to enhance service.

Conclusion

- Project Summary
  - Developed a modern, full-stack Library Management System using React + JavaScript (frontend) and Spring Boot 3 + MySQL (backend).
  - The application enables members to browse, borrow, reserve, and renew books while administrators gain full control over catalog, users, loans, reservations, and fines – all with real-time updates via React Query, role-based access, and a clean, responsive UI.
  - Well tested and designed to mirror real-world library workflows
- Key Achievements
  - Met business objectives and delivered the application well within project timeline
  - Met all business rules set out in the BRD
  - Met all functional requirements set out in the BRD/SRS within the require technical stack
- Q & A